

Git, GitHub et GitQuest

Du workflow Fork & Pull Request au visualiseur d'arbres Git en Python

Projet tutoré — BUT Science des Données (1 ère année)

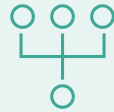
Réalisé par : M'mahawa doss camara

Concepts fondamentaux



Dépôt

Espace qui stocke les fichiers et tout l'historique des modifications. Local sur la machine, ou distant sur GitHub.



Fork

Copie indépendante d'un dépôt existant sur son propre compte GitHub, sans toucher à l'original.



Branche

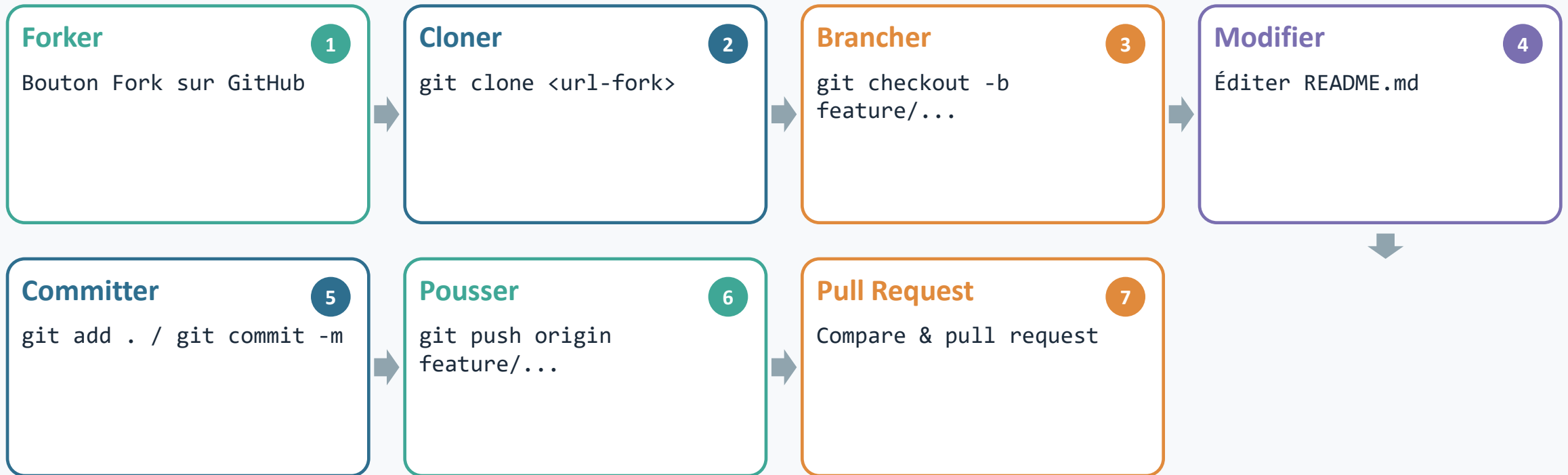
Ligne de développement isolée pour travailler sans impacter la branche principale (main).



Pull Request

Demande formelle d'intégrer ses modifications dans le dépôt d'origine, avec relecture.

Workflow complet : Fork → Pull Request



GitQuest — visualiseur d'arbres Git

- **Outil Python développé pour le projet**

Visualise l'arbre des commits d'un dépôt Git, branches et merges inclus.

- **7 modules, responsabilités séparées**

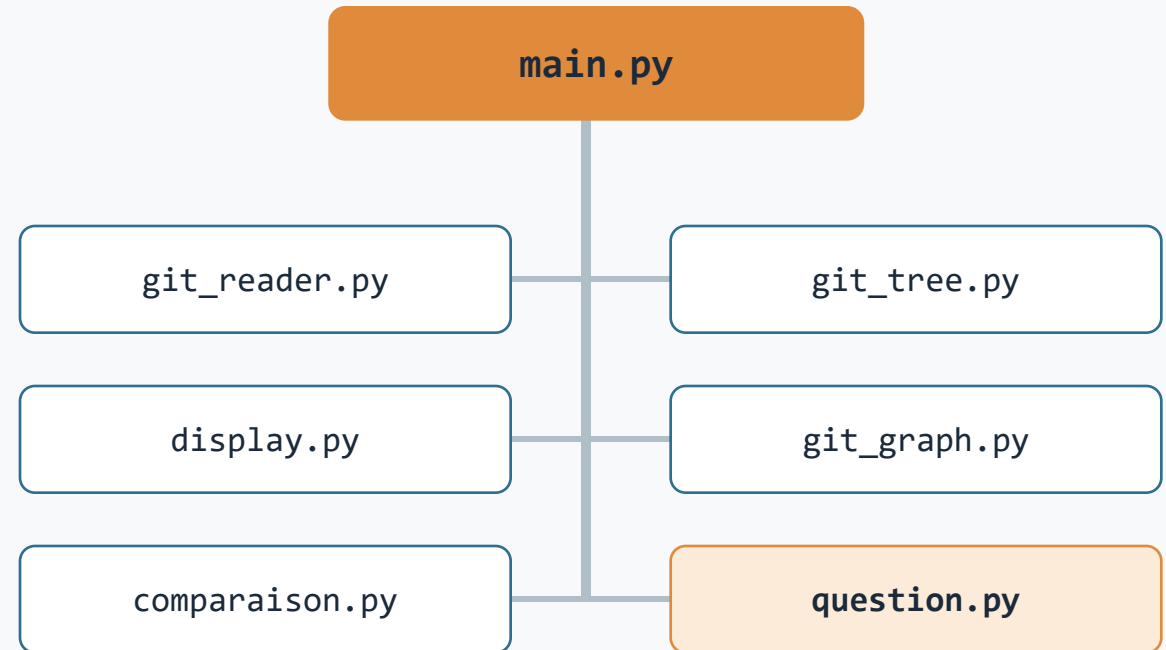
main.py orchestre git_reader, git_tree, display, git_graph, comparaison, question.

- **5 modes principaux**

Démo, lecture d'un dépôt, mode interactif, visualisation matplotlib, outil enseignant.

- **Mode pédagogique**

L'étudiant voit côte à côte l'état actuel et l'état cible à atteindre.



Fonctionnalité ajoutée : créer ses propres questions

Enseignant



- Préparer un dépôt dans l'état cible (branches, commits, tags)
- Exporter cet état en fichier JSON
- Distribuer le JSON aux étudiants

```
python main.py <depot> \  
  --save-target q1.json
```

Étudiant



- Reçoit le fichier JSON de l'enseignant
- Lance le mode interactif avec la question
- Voit côte à côte état actuel et état cible

```
python main.py <depot> --loop \  
  --target q1.json
```

GitQuest en action

Aperçu de l'outil dans ses deux modes d'affichage principaux.

```
$ python main.py
+-----+-----+-----+
| main   |feature | hotfix |
+-----+-----+-----+
| * c3   | * c5   |         |
| * c2 --+- * c4 | * c6   |
| * c1 --+-----+----- |
+-----+-----+-----+
```

Mode démo — arbre encadré en 3 colonnes (terminal)

```
--loop --target q1.json
ETAT ACTUEL | ETAT CIBLE
----- | -----
* main: c2  | * main: c2
            | * feat: c3
>>> git commit | [v] resolu
>>> ...       | objectif OK
```

Mode interactif — état actuel vs état cible côte à côte

Récapitulatif des commandes Git

Commande	Rôle
<code>git clone <url></code>	Télécharge un dépôt distant sur la machine
<code>git checkout -b <nom></code>	Crée une branche et bascule dessus
<code>git add <fichier></code>	Ajoute un fichier à l'index (staging)
<code>git commit -m "msg"</code>	Enregistre les modifications dans l'historique
<code>git push origin <branche></code>	Envoie les commits vers le dépôt distant
<code>git pull</code>	Récupère et intègre les modifications distantes
<code>git log --oneline --graph</code>	Affiche l'historique sous forme d'arbre
<code>git branch</code>	Liste les branches locales
<code>git status</code>	Affiche l'état du répertoire de travail
<code>git diff</code>	Montre les différences non commit

Résultats et apprentissages

Résultats obtenus

- ✓ Workflow Fork / Clone / Branch / Push / PR maîtrisé
- ✓ Git installé et configuré sous Windows (PowerShell)
- ✓ Outil Python GitQuest fonctionnel — 7 modules
- ✓ Tri topologique (Kahn) pour l'ordre des commits
- ✓ Compareur indépendant des hashes
- ✓ Pull Request soumise sur le dépôt d'origine

Compétences développées

- ✓ Gestion de version et collaboration via GitHub
- ✓ Python orienté objet, séparation des responsabilités
- ✓ Algorithmique : tri topologique, comparaison d'arbres
- ✓ Sérialisation JSON pour la persistance
- ✓ Communication technique : docstrings, README, rapport

Conclusion

Au-delà d'une simple initiation au workflow Git/GitHub, ce projet a mis en pratique la dynamique de contribution open source : du fork à la Pull Request, jusqu'à une fonctionnalité enseignant répondant à un besoin réel.

Des apprentissages directement transposables en alternance et en projet, où Git est omniprésent.

Perspectives : conflits de merge en mode interactif, interface web légère, packaging de GitQuest.